

Pod Security Admission e Security Context

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Descrivere i tre livelli di Pod Security Admission (PSA): **privileged**, **baseline**, **restricted**
- Applicare i label PSA a un namespace per enforcing, warning o audit
- Configurare il **securityContext** a livello Pod e container secondo le best practice di produzione
- Usare **runAsNonRoot**, **allowPrivilegeEscalation: false**, **readOnlyRootFilesystem** e **capabilities**
- Spiegare la coesistenza di PSA e SCC su OpenShift 4.11+
- Comprendere il ruolo di SELinux su Red Hat CoreOS in ARO
- Applicare un Security Context completo che soddisfi sia **restricted-v2** SCC che PSA **restricted**

Teoria e Concetti Chiave

Pod Security Admission (PSA)

Pod Security Admission è un Admission Controller built-in di Kubernetes (stabile dalla 1.25) che implementa i **Pod Security Standards**. A differenza delle SCC (OpenShift-specific), PSA è disponibile su qualsiasi distribuzione Kubernetes upstream.

I Tre Livelli PSA

Livello	Descrizione	Tipico ambiente
privileged	Nessuna restrizione — permette tutto	Namespace di sistema (kube-system , openshift-*)
baseline	Restrizioni minime — blocca le escalation note più pericolose	Workload generici, ambienti non critici
restricted	Massima sicurezza — segue le best practice correnti	Produzione, workload sensibili

Modalità PSA

Per ogni livello si può scegliere una modalità:

Modalità	Effetto
enforce	Rifiuta i Pod che violano il livello
audit	Registra le violazioni nei log di audit, ma non rifiuta
warn	Visualizza un warning all'utente, ma non rifiuta

Label di Namespace per PSA

```
# Applicare PSA restricted in modalità enforce al namespace myproject
kubectl label namespace myproject \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/enforce-version=latest

# Aggiungere warning per segnalare violazioni senza bloccare
kubectl label namespace myproject \
  pod-security.kubernetes.io/warn=restricted \
  pod-security.kubernetes.io/audit=restricted
```

SCC vs PSA: Coesistenza su OpenShift 4.11+

Su OpenShift 4.11+, **entrambi** i sistemi sono attivi contemporaneamente:

Caratteristica	SCC (OpenShift)	PSA (Kubernetes)
Disponibilità	Solo OpenShift	Kubernetes nativo
Scope	Cluster-scoped, assegnato tramite SA/utente	Namespace-scoped, configurato con label
Granularità	Alta (UID, fsGroup, SELinux, volumi, capabilities)	Media (standard predefiniti con 3 livelli)
Mutazione	Può modificare il Pod (aggiunge fsGroup, SELinux, ecc.)	Solo validazione — non modifica il Pod
Priorità in OCP	SCC valida prima	PSA valida dopo SCC

 **Suggerimento:** In OpenShift, le SCC sono il meccanismo primario. PSA in modalità **warn** è utile per identificare workload non conformi, ma in produzione su ARO è la SCC che governa effettivamente l'esecuzione del Pod.

Security Context a Livello Pod

Il `securityContext` a livello `spec.securityContext` si applica a tutti i container del Pod:

Campo	Descrizione	Best practice produzione
<code>runAsNonRoot</code>	Rifiuta container che girano come root (UID 0)	<code>true</code>
<code>runAsUser</code>	UID del processo principale	Valore specifico dell'app (es. 1001)
<code>runAsGroup</code>	GID del processo principale	Valore specifico dell'app
<code>fsGroup</code>	GID per i volumi montati (ownership automatica)	Valore specifico dell'app
<code>fsGroupChangePolicy</code>	Quando applicare l'ownership ai volumi	<code>OnRootMismatch</code> (performance)
<code>seccompProfile</code>	Profilo seccomp (syscall filtering)	<code>type: RuntimeDefault</code>
<code>supplementalGroups</code>	GID aggiuntivi per il processo	Solo se necessario

Security Context a Livello Container

Il `securityContext` a livello `containers[].securityContext` si applica al singolo container (override di quello del Pod):

Campo	Descrizione	Best practice produzione
<code>allowPrivilegeEscalation</code>	Permette al processo di acquisire più privilegi del parent (es. setuid)	<code>false</code>
<code>readOnlyRootFilesystem</code>	Filesystem root del container è read-only	<code>true</code>
<code>capabilities.drop</code>	Lista di Linux capabilities da rimuovere	<code>["ALL"]</code>
<code>capabilities.add</code>	Lista di Linux capabilities da aggiungere (dopo drop ALL)	Solo il minimo necessario
<code>privileged</code>	Container completamente privilegiato (accesso kernel)	<code>false</code> sempre
<code>runAsNonRoot</code>	Override del valore Pod-level	<code>true</code>
<code>seccompProfile</code>	Override del profilo seccomp	<code>type: RuntimeDefault</code>

Linux Capabilities Comuni

Capability	Descrizione	Quando aggiungere
<code>NET_BIND_SERVICE</code>	Bind a porte < 1024	Server HTTP sulla porta 80
<code>CHOWN</code>	Cambia ownership dei file	Script di init che cambiano owner
<code>DAC_OVERRIDE</code>	Bypassa i controlli di permesso file	Raramente — da evitare
<code>SYS_PTRACE</code>	Debugging processi	Solo debug — mai produzione

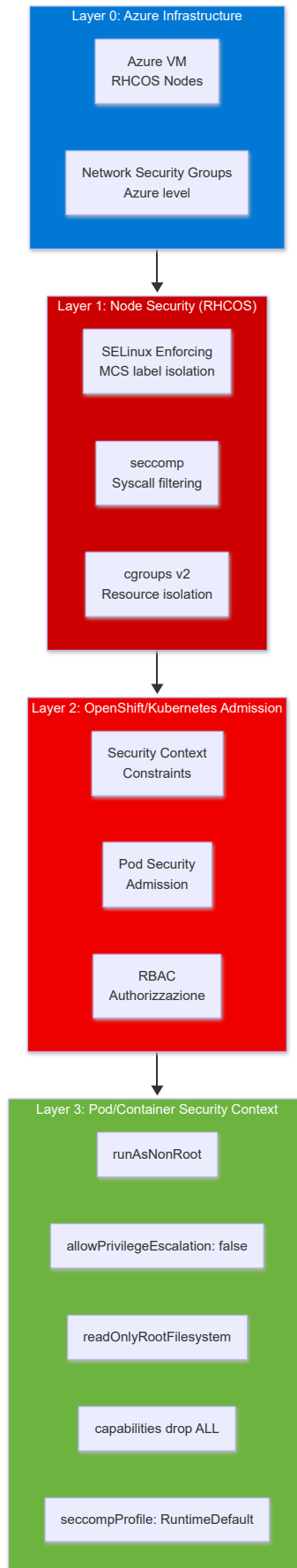
SELinux Context

Su Red Hat CoreOS (i nodi di ARO), **SELinux** è attivo in modalità **enforcing**. OpenShift gestisce automaticamente le label SELinux tramite SCC (`seLinuxContext.type: MustRunAs`), assegnando un contesto **MCS (Multi-Category Security)** univoco a ogni Pod per isolamento:

```
# SELinux context nel security context del Pod
securityContext:
  seLinuxOptions:
    level: "s0:c123,c456"    # ← MCS label assegnata automaticamente da OpenShift
    # Non specificare manualmente a meno di requisiti specifici
```

Diagrammi

Strati di Sicurezza di un Pod su OpenShift/ARO



Comandi Pratici con Output Atteso

```
# Applicare PSA restricted al namespace myproject (warn prima di enforce)
oc label namespace myproject \
  pod-security.kubernetes.io/warn=restricted \
  pod-security.kubernetes.io/warn-version=latest

# Output atteso:
# namespace/myproject labeled
```

```
# Verificare le label PSA di un namespace
oc get namespace myproject -o jsonpath='{.metadata.labels}' | jq

# Output atteso:
# {
#   "kubernetes.io/metadata.name": "myproject",
#   "pod-security.kubernetes.io/warn": "restricted",
#   "pod-security.kubernetes.io/warn-version": "latest"
# }
```

```
# Testare se un manifest è conforme a PSA restricted prima del deploy
oc apply --dry-run=server -f pod-manifest.yaml -n myproject

# Output atteso (se non conforme):
# Warning: would violate PodSecurity "restricted:latest":
#   allowPrivilegeEscalation != false (container "app" must set
# securityContext.allowPrivilegeEscalation=false),
#   unrestricted capabilities (container "app" must set
# securityContext.capabilities.drop=["ALL"])
```

```
# Creare un Pod conforme a restricted-v2 SCC e PSA restricted
oc run secure-pod \
  --image=registry.access.redhat.com/ubi9/ubi-minimal:9.4 \
  --command -- sleep 3600 \
  -n myproject

# Verificare la SCC applicata
oc get pod secure-pod -n myproject \
  -o jsonpath='{.metadata.annotations.openshift\.io/scc}'

# Output atteso:
# restricted-v2
```

```
# Verificare lo stato SELinux sui nodi ARO (RHCOS)
oc debug node/<node-name> -- chroot /host getenforce
```

```
# Output atteso:
# Enforcing
```

```
# Visualizzare il security context effettivo di un Pod in esecuzione
oc get pod myapp-7f9d8b-abc12 -n myproject \
  -o jsonpath='{.spec.securityContext}' | jq
```

```
# Output atteso:
# {
#   "fsGroup": 1000680000,
#   "seLinuxOptions": {"level": "s0:c26,c15"},
#   "seccompProfile": {"type": "RuntimeDefault"}
# }
```

```
# Verificare le syscall bloccate dal profilo seccomp (nei log di audit SELinux)
oc debug node/<node-name> -- \
  chroot /host ausearch -m seccomp --start recent
```

```
# Output atteso:
# type=SECCOMP msg=audit(1716547200.123:456): auid=... syscall=323 compat=0 ip=...
# code=KILL_THREAD
```

Esempi YAML Completi

Pod Compliant con `restricted-v2` SCC e PSA `restricted`

```
# Pod con Security Context completo per produzione
# Conforme a: SCC restricted-v2 (OpenShift) + PSA restricted (Kubernetes)
apiVersion: v1
kind: Pod
metadata:
  name: secure-app
  namespace: myproject
  labels:
    app: secure-app
spec:
  # ServiceAccount senza mount automatico del token
  serviceAccountName: myapp-sa
  automountServiceAccountToken: false

  # Security Context a livello POD (si applica a tutti i container)
  securityContext:
    runAsNonRoot: true           # ← rifiuta il Pod se il container gira come
    UID 0
    runAsUser: 1001              # ← UID fisso per il processo principale
    runAsGroup: 1001            # ← GID del processo principale
    fsGroup: 1001                # ← GID per i volumi persistenti (ownership
    automatica)
    fsGroupChangePolicy: OnRootMismatch # ← applica chown solo se necessario
    (performance)
    seccompProfile:
      type: RuntimeDefault        # ← profilo seccomp del runtime container
    (consigliato)
    # SELinux: lasciare che OpenShift assegni automaticamente il contesto MCS
    # selinuxOptions:             # ← non specificare manualmente

  containers:
    - name: app
      image: registry.access.redhat.com/ubi9/ubi-minimal:9.4
      command: ["/bin/sh", "-c", "sleep 3600"]

  # Security Context a livello CONTAINER (override del Pod-level se necessario)
  securityContext:
    allowPrivilegeEscalation: false # ← blocca sudo, setuid, newuidmap ecc.
    readOnlyRootFilesystem: true    # ← filesystem root in sola lettura
    runAsNonRoot: true               # ← ridondante ma esplicito
    privileged: false                # ← mai true in produzione
    capabilities:
      drop:
        - ALL                       # ← rimuove TUTTE le Linux capabilities
      # add:                         # ← aggiungere SOLO se strettamente
      necessario
      # - NET_BIND_SERVICE           # ← esempio: se il server gira sulla porta
```


80

```

resources:
  requests:
    memory: "128Mi"
    cpu: "100m"
  limits:
    memory: "256Mi"
    cpu: "500m"

# Volume per file temporanei (readOnlyRootFilesystem richiede emptyDir per
/tmp)
volumeMounts:
- name: tmp-dir
  mountPath: /tmp # ← necessario se il processo scrive in
/tmp
- name: app-config
  mountPath: /etc/app
  readOnly: true # ← sempre readOnly per ConfigMap e Secret

volumes:
- name: tmp-dir
  emptyDir:
    medium: Memory # ← tmpfs in memoria (più sicuro: non
persiste su disco)
    sizeLimit: 64Mi # ← limita il consumo di memoria per /tmp
- name: app-config
  configMap:
    name: myapp-config

```

Namespace con PSA Configurato

```

# Namespace con PSA in modalità warn + audit per identificare non-conformità
# PRIMA di abilitare enforce
apiVersion: v1
kind: Namespace
metadata:
  name: myproject
  labels:
    # PSA warn: avvisa l'utente ma non blocca
    pod-security.kubernetes.io/warn: restricted
    pod-security.kubernetes.io/warn-version: latest
    # PSA audit: registra nei log di audit
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/audit-version: latest
    # PSA enforce: blocca i Pod non conformi (abilitare dopo verifica)
    # pod-security.kubernetes.io/enforce: restricted
    # pod-security.kubernetes.io/enforce-version: latest

```

Note Specifiche per Azure ARO

SELinux su Red Hat CoreOS (RHCOS)

I nodi ARO usano **Red Hat CoreOS** con SELinux in modalità **enforcing**. OpenShift gestisce automaticamente le label SELinux MCS per ogni Pod, garantendo isolamento tra container anche se girano con lo stesso UID:

```
# Verificare il contesto SELinux di un processo nel container
oc exec myapp-7f9d8b-abc12 -n myproject -- \
  cat /proc/1/attr/current

# Output atteso:
# system_u:system_r:container_t:s0:c26,c15
# ↑ type container_t + label MCS univoca s0:c26,c15
```

 **Suggerimento:** Su ARO non è necessario configurare manualmente il contesto SELinux per la maggior parte dei workload. OpenShift lo gestisce tramite la SCC. Specificare **seLinuxOptions** manualmente solo per workload che accedono a volumi hostPath con label SELinux specifiche.

Microsoft Defender for Containers su ARO

Microsoft Defender for Containers analizza le immagini container e i Pod in esecuzione su ARO per vulnerabilità e non-conformità di sicurezza:

```
# Verificare lo stato di Microsoft Defender for Containers su ARO
az security pricing show \
  --name Containers \
  --query pricingTier -o tsv

# Output atteso:
# Standard

# Verificare le raccomandazioni di sicurezza per il cluster ARO
az security assessment list \
  --query "[?contains(displayName,'Kubernetes')]" \
  -o table
```

Verifica Conformità Security Context su ARO

```
# Identificare tutti i Pod nel cluster che girano come root (UID 0)
oc get pods --all-namespaces \
  -o json | jq -r '
    .items[] |
    select(
      .spec.securityContext.runAsUser == 0 or
      .spec.containers[].securityContext.runAsUser == 0
    ) |
    "\(.metadata.namespace)/\(.metadata.name)'"

# Identificare Pod senza readOnlyRootFilesystem
oc get pods --all-namespaces \
  -o json | jq -r '
    .items[] |
    select(
      .spec.containers[].securityContext.readOnlyRootFilesystem != true
    ) |
    "\(.metadata.namespace)/\(.metadata.name)'" | \
  grep -v "^openshift-"
```

Riepilogo dei Punti Chiave

- **PSA** è il meccanismo Kubernetes upstream con tre livelli (**privileged**, **baseline**, **restricted**) e tre modalità (**enforce**, **warn**, **audit**); si configura con label sul namespace
 - Su OpenShift 4.11+, PSA e SCC **coesistono**: la SCC è il meccanismo primario e più granulare; PSA fornisce un secondo livello di validazione
 - Il **securityContext** a livello Pod si applica a tutti i container; quello a livello container fa override selettivo
 - Le best practice di produzione per il Security Context: **runAsNonRoot: true**, **allowPrivilegeEscalation: false**, **readOnlyRootFilesystem: true**, **capabilities.drop: ["ALL"]**, **seccompProfile: RuntimeDefault**
 - Con **readOnlyRootFilesystem: true**, montare **emptyDir** per le directory in scrittura necessarie (es. **/tmp**)
 - Su ARO (RHCOS), **SELinux** è sempre **enforcing**; OpenShift assegna automaticamente label MCS univoche per ogni Pod garantendo isolamento
 - **Microsoft Defender for Containers** fornisce vulnerability scanning su immagini e runtime su ARO
-

Domande di Verifica

1. Quale label di namespace attiva PSA in modalità **enforce** al livello **restricted**?

- A) `security.kubernetes.io/policy=restricted`
- B) `pod-security.kubernetes.io/enforce=restricted` ✓
- C) `openshift.io/scc=restricted-v2`
- D) `kubernetes.io/psp=restricted`

2. Un container con **readOnlyRootFilesystem: true** deve scrivere file di log temporanei. Quale è la soluzione corretta?

- A) Disabilitare `readOnlyRootFilesystem`
- B) Usare una SCC con `allowWriteFilesystem: true`
- C) Montare un `emptyDir` nella directory di log e scrivere lì ✓
- D) Usare `hostPath` per scrivere direttamente sul nodo

3. Cosa fa **capabilities.drop: ["ALL"]** nel Security Context di un container?

- A) Rimuove il container dal namespace delle capability Linux del kernel
- B) Rimuove tutte le Linux capabilities dal container (equivalente a un processo non-privilegiato) ✓
- C) Disabilita SELinux per il container
- D) Imposta `privileged: false` per il container

4. Su OpenShift 4.11+, quale affermazione descrive correttamente la relazione tra SCC e PSA?

- A) PSA ha sostituito completamente le SCC a partire da OpenShift 4.11
- B) Le SCC vengono validate dopo PSA
- C) PSA e SCC coesistono: la SCC è il meccanismo primario, PSA fornisce validazione aggiuntiva ✓
- D) PSA è disabilitato di default su OpenShift e va attivato manualmente

5. Su ARO con Red Hat CoreOS, quale affermazione riguardo SELinux è corretta?

- A) SELinux è disabilitato di default su ARO e va attivato manualmente
- B) SELinux è attivo in modalità `permissive` (solo log, no blocchi)
- C) SELinux è attivo in modalità `enforcing`; OpenShift assegna automaticamente label MCS univoche per ogni Pod ✓
- D) SELinux usa solo label `container_t` senza distinzione tra Pod (nessun isolamento MCS)